

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-ad6d53cc935c97d77.jsonl`  
- SHA-256: `f0ea370ec4b8ebc80225841c2094d8b64d78b1ca20e4e7e818cd50e1aeb60acd`  
- Source modified: `2026-07-07T15:24:56+00:00`  
- Imported at: `2026-07-09T08:32:24+00:00`  
- project: `wf_a8f43370-4cb`  
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:18:57.521Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):  
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,  
    fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote  
  the  
    choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-  
  backed  
    AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-  
  process as  
    before). Plus 4 tests in tests/test_api_cloud_dispatch.py.  
- SPA (repo undefined, branch undefined, PR #undefined):  
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when  
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).  
Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)  
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)  
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)  
- undefined/spa.diff (SPA commit diff)  
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;  
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;  
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);  
and under undefined: web/src/api/types.ts, web/src/api/client.ts,  
web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

Adversarially VERIFY this finding from the 'weights-resolution-fidelity' review. Your job is to REFUTE it if at all possible: read the ACTUAL authoritative code (the snapshot files above and any repo files needed) and determine whether the described failure REALLY occurs against the real code as written. Default to REFUTED unless you can reproduce it concretely. Beware findings that assume a code path that does not exist, or ignore an earlier guard.

FINDING:

- severity: medium
- category: weights-resolution-fidelity
- location: backend/orchestration.py ? _local_detector_weights_present() tracknet branch: `if name == "tracknet\_hybrid" or (name == "fusion\_hybrid" and substrate == "tracknet")`
- claim: For tracknet_hybrid and fusion_hybrid(substrate="tracknet") the guard's "weights present ? run in-process locally" escape hatch is UNSOUND on the hosted control-plane: it keys on indexing.tracknet.weights_path existence, but there is NO native TrackNet runner and Linux has no WSL, so weights-present does not imply the in-process detector can run. The docstring's claim to "mirror the native/WSL runners' resolution" is itself false for tracknet on Linux.
- failure_scenario: Hosted Linux CP (deployment.compute_target=cloud_run ? _cloud_available True). Operator sets indexing.tracknet.weights_path to a path that exists on the CP (any existing file flips the check to True). POST /api/process segmenter=tracknet_hybrid compute=local ? _local_detector_weights_present returns True ? the choice=="local" fast-fail is SKIPPED ? the in-process run proceeds and dies deep: with detector_impl=native the base _build_native_runner raises NotImplementedError ("detector_impl='native' is not supported for the 'TrackNet' family"), or with detector_impl=auto the WSL TrackNetRunner.healthcheck runs `wsl` ? FileNotFoundError ("`wsl` not found"). Result is exactly the cryptic deep-failure UX the fix set out to eliminate for one of the three segmenters it explicitly claims to cover, instead of the actionable "Choose 'Run in cloud'." (Same for fusion_hybrid with substrate=tracknet.) Note: the DEFAULT indexing.tracknet.weights_path="" makes the guard return False and fire correctly, so this only misfires with a configured, existing tracknet weights_path; it is a coverage/fidelity gap, not a regression versus pre-fix.
- suggested_fix: Make "is there a runner that can run this detector locally on THIS box" part of the predicate, independent of weights. Since there is no native TrackNet runner, for tracknet_hybrid and fusion(substrate=tracknet) return False (not-locally-runnable) on a cloud-capable server so the fast-fail always fires, regardless of indexing.tracknet.weights_path ? or gate on the real precondition (detector_impl / WSL availability) the way trajectory_hybrid._resolve_runner does.

Return CONFIRMED only with a concrete repro grounded in the real code; else REFUTED with the reason.

2. assistant (2026-07-07T15:19:05.622Z)

I'll start by locating the authoritative snapshot files and the actual repo code, since the paths show "undefined" placeholders.

3. user (2026-07-07T15:19:09.061Z)

total 62

```
drwxr-xr-x 1 avidu 197609  0 Jul  5 12:57 .
drwxr-xr-x 1 avidu 197609  0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609  0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609  0 Jul  7 20:38 .git
-rw-r--r-- 1 avidu 197609 297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609  0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609 323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609  0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609  0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609  0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609  0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
```

```

drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web
===
total 108
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609  905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:49 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport

```

4. user (2026-07-07T15:19:10.114Z)

```

---
/c/Users/avidu/AppData/Local/Temp/backend.diff
---
```

5. assistant (2026-07-07T15:19:13.640Z)

Snapshot files are in the Temp directory. Let me locate all of them and the real repo.

6. user (2026-07-07T15:19:16.749Z)

```

total 702
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:49 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 25 23:31 .claude
-rw-r--r-- 2 avidu 197609 53248 Jul  7 20:49 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:44 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609    0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609   6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609   2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:43 docs
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609 1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:48 output
-rw-r--r-- 1 avidu 197609   5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609    240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609   1674 Jul  2 20:05 requirements.txt

```

```
-rw-r--r-- 1 avidu 197609 585 Jul 2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609 809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609 2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609 4594 Jun 29 13:47 run_regression.py
-rwxr-xr-x 1 avidu 197609 5073 Jul 7 10:27 score_golden_gate.py
-rwxr-xr-x 1 avidu 197609 7617 Jul 7 03:32 score_local.py
```

7. user (2026-07-07T15:19:16.800Z)

```
ls: cannot access '/c/Users/avidu/AppData/Local/Temp/claude-5b6d-cwd': No such file or directory
-rw-r--r-- 1 avidu 197609 11027 Jul 7 20:34 backend.diff
-rw-r--r-- 1 avidu 197609 5007 Jul 7 20:34 spa.diff
```

8. assistant (2026-07-07T15:19:21.070Z)

Only the diffs are in Temp root. Let me find the full snapshot files.

9. user (2026-07-07T15:19:22.896Z)

```
1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date: Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
17         weights for the requested detector aren't readable, a forced compute="local"
18         now fails FAST with an actionable message ("this hosted server can't run
19         detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20         compute="cloud"-not-available reject ? instead of the silent in-process trap.
21         A truly-local box, or a cloud-wired box that actually has the weights on disk,
22         is unaffected (the weights-present check is the escape hatch); a weight-free
23         segmenter (motion) is out of scope. Honest provenance: path=not_run,
24         requested=local, dispatched=False.
25
26         Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27         hatch, motion unaffected, pure-local not guardrailed).
28
29         Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31         diff --git a/backend/orchestration.py b/backend/orchestration.py
32         index 5e9f510..bfe39f5 100644
33         --- a/backend/orchestration.py
34         +++ b/backend/orchestration.py
35         @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36             return n
37
38
39         += Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
40         box. A hosted
41         += cloud-serving control-plane bakes NO detector weights into its image (only the worker
42         Cloud Run Job
43         += GCSFuse-mounts them), so forcing compute="local" for one of these there can never
44         succeed ? the
```

```

42     ## guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
43     ## "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
44     +_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
45     +
46     +
47     +def _local_detector_weights_present(cfg, seg_name: str) -> bool:
48     +     """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
49     +     on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
50     +     and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
51     +     otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
52     +     A non-weights segmenter returns True (nothing local to check)."""
53     +     name = (seg_name or "").strip().lower()
54     +     if name not in _WEIGHTS_BACKED_SEGMENTERS:
55     +         return True
56     +     idx = getattr(cfg, "indexing", None)
57     +     substrate = (
58     +         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
59     +     ).strip().lower()
60     +     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
61     +         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
62     +     else: # wasb_hybrid, or fusion on the (default) WASB substrate
63     +         path = (
64     +             os.environ.get("WASB_WEIGHTS")
65     +             or getattr(getattr(idx, "wasb", None), "weights_path", "")
66     +             or ""
67     +         )
68     +     return bool(path) and os.path.exists(os.path.expanduser(path))
69     +
70     +
71     def _cloud_available(cfg) -> bool:
72     +     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
73
74     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75     +
76     +     choice = ""
77     +     if choice == "local":
78     +         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79     +         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80     +         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81     +         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82     +         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83     +         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84     +         # below exactly as before (the weights-present check is the escape hatch).
85     +         if (
86     +             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87     +             and _cloud_available(cfg)
88     +             and not _local_detector_weights_present(cfg, seg_name)
89     +         ):
90     +             return _failed(
91     +                 f"this hosted server can't run detection locally ? the '{seg_name}'

```

```

model weights "
92     +           "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93     +           False,
94     +           )
95     return None # explicit "run on my machine" ? in-process regardless of the
server's target
96     if choice == "cloud":
97         if not _cloud_available(cfg):
98             diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99             index 8382234..7993b7f 100644
100            --- a/tests/test_api_cloud_dispatch.py
101            +++ b/tests/test_api_cloud_dispatch.py
102            @@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
103                assert all(_meta(s)["source"] == "cloud_run" for s in segs)
104
105
106     ## --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-----
107     ## A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
108     ## them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
109     ## FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
110     ## that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
111     +
112     +
113     +def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114     +     """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
115     +     a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
116     +     _db, _bucket, _mp = cloud_env
117     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
118     +     m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
119     +     fake = _FakeCloudClient()
120     +     _inject_fake(monkeypatch, fake)
121     +
122     +     out = m._execute_processing(
123     +         m.ProcessRequest(
124     +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
125     +         )
126     +     )
127     +     assert out["status"] == "failed"
128     +     assert "can't run detection locally" in out["message"]
129     +     assert "Run in cloud" in out["message"]
130     +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
131     +     assert fake.calls == [] # never dispatched
132     +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
133     +     assert out["compute"]["requested"] == "local"
134     +     assert out["compute"]["dispatched"] is False
135     +     assert out["compute"]["path"] == "not_run"
136     +
137     +
138     +def test_local_on_cloud_server_with_weights_present_runs_in_process(
139     +     cloud_env, tmp_path, monkeypatch
140     +):
141     +     """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
142     +     detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""

```

```

143 + _db, _bucket, _mp = cloud_env
144 + weights = tmp_path / "wasb_badminton_best.pth.tar"
145 + weights.write_bytes(b"WEIGHTS")
146 + monkeypatch.delenv("WASB_WEIGHTS", raising=False)
147 + m.global_config.indexing.wasb.weights_path = str(weights)
148 + monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
149 + fake = _FakeCloudClient()
150 + _inject_fake(monkeypatch, fake)
151 +
152 + out = m._execute_processing(
153 +     m.ProcessRequest(
154 +         video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
155 +     )
156 + )
157 + assert out["status"] == "success"
158 + assert fake.calls == [] # in-process, not dispatched
159 + assert out["compute"]["path"] == "in_process"
160 +
161 +
162 +def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
163 +    cloud_env, monkeypatch
164 +):
165 +    """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
166 +    compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
167 +    _db, _bucket, _mp = cloud_env
168 +    m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
169 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
170 +    fake = _FakeCloudClient()
171 +    _inject_fake(monkeypatch, fake)
172 +
173 +    out = m._execute_processing(
174 +        m.ProcessRequest(
175 +            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
176 +        )
177 +    )
178 +    assert out["status"] == "success"
179 +    assert fake.calls == []
180 +    assert out["compute"]["path"] == "in_process"
181 +
182 +
183 +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184 +    tmp_path, monkeypatch
185 +):
186 +    """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187 +    weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188 +    'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189 +    db = Database(str(tmp_path / "t.db")) # noqa: F841
190 +    monkeypatch.setattr(m, "db_instance", db)
191 +    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192 +    monkeypatch.setattr(m, "global_config", cfg)
193 +    monkeypatch.setattr(
194 +        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
195 +    )
196 +    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197 +    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198 +    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199 +    cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201 +    video = tmp_path / "v.mp4"

```

```

202 + video.write_bytes(b"hashable-bytes")
203 +
204 + out = m._execute_processing(
205 +     m.ProcessRequest(
206 +         video_path=str(video), segmenter="wasb_hybrid", compute="local"
207 +     )
208 + )
209 + # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
210 + assert out["status"] == "success"
211 + assert out["compute"]["path"] == "in_process"
212 +
213 +
214 + # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

10. user (2026-07-07T15:19:24.459Z)

(Bash completed with no output)

11. assistant (2026-07-07T15:19:29.726Z)

The full snapshot files aren't present; I'll work from the diff plus the actual repo. Let me check the real repo's orchestration.py to see if it has the change, and gather ground-truth on the runners.

12. user (2026-07-07T15:19:35.876Z)

```

/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/bhi-fix-wt/backend/orchestration.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/bhi-fix-
wt/tests/test_api_cloud_dispatch.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_post.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_new.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_post.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt/backend/orchestration.py
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-
wt/backend/__pycache__/orchestration.cpython-313.pyc
/c/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt/tests/test_api_cloud_dispatch.py
=== grep in real repo ===

```

13. user (2026-07-07T15:19:36.784Z)

```

3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag
(#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed +

```

```

downsample fix proven (#504)
=== branch ===
fix/cloud-run-exec-status-fastfail
=== status ===
M backend/compute/__init__.py
M backend/compute/base.py
M backend/compute/cloud_run.py
M backend/worker/__main__.py
M tests/test_compute_backend.py
?? score_golden_gate.py
?? score_local.py

```

14. assistant (2026-07-07T15:20:19.469Z)

The authoritative snapshots are in my scratchpad. The real repo doesn't have the change yet, so `orchestration_new.py` is authoritative for the fix; the real repo is ground truth for `runners/config`. Let me read the key files.

15. user (2026-07-07T15:20:21.082Z)

```

150             ) # OverflowError on 'inf'/'1e400'
151             except (ValueError, OverflowError):
152                 meta["shot_count"] = sc
153                 db.add_play_segment(video_id, start, end, metadata=meta)
154                 n += 1
155             return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")

```

```

184         or getattr(getattr(idx, "wasb", None), "weights_path", "")
185         or ""
186     )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
192         3).
193         True when the control plane is wired for Cloud Run: either the configured default
194         target is
195         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
196         coordinates
197         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
198         (``storage.output_root``)
199         are present so a forced per-request override can actually dispatch + collect a
200         result.
201         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
202         would work."""
203         deployment = getattr(cfg, "deployment", None)
204         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
205             return True
206         has_job = bool(
207             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
208         )
209         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
210         return bool(has_job and out_root)
211
212     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
213         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
214         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
215         comes
216         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
217         "starting"
218         (queue/cold-start), so the very first tick already shows movement."""
219         minutes = int(elapsed_sec) // 60
220         if minutes < 1:
221             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
222         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
223
224     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
225         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
226         gs://
227         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
228         keep
229         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
230         no
231         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
232         if getattr(input_ref, "backend", "") != "gcs":
233             return None
234         sc = _storage_settings(cfg)
235         if not sc:
236             return None
237         try:
238             st = storage.get_storage("gcs", config=sc).stat(input_ref)
239             return getattr(st, "md5", None) or None
240         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
241             logger.info(
242                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
243                 getattr(input_ref, "uri", input_ref),
244                 exc_info=True,

```

```

238         )
239         return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
246         for
247         both producers (the drain's DB-cache short-circuit and the submit-time cache,
248         #484)."""
249         return {
250             "video_id": video_id,
251             "status": "success",
252             "message": (
253                 f"Reused {len(segments)} cached play segments. "
254                 "Pass force=true to reprocess this video."
255             ),
256             "results": validation_results or [],
257             "play_segments": segments,
258             "cached": True,
259             "compute": {"path": "cached", "requested": requested_compute},
260         }
261
262     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
263         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
264         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
265         decide."""
266         ref = storage.parse_uri(uri)
267         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
268             local = storage.fetch(
269                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
270             )
271             with open(local, encoding="utf-8") as f:

```

16. user (2026-07-07T15:20:23.524Z)

```

__init__.py
__pycache__
audio_features.py
base.py
court_gate.py
device.py
fusion_features.py
native_wasb_runner.py
person_detector.py
rally_gate.py
stub_runner.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
=== segmenters ===
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py

```

17. assistant (2026-07-07T15:20:30.287Z)

Confirmed: there's a `native\_wasb\_runner.py` but no `native\_tracknet\_runner.py`. Now let me examine the tracknet runner, the segmenter build path, and the config model.

18. user (2026-07-07T15:20:30.544Z)

```
1      """TrackNet-based hybrid play segmenter.
2
3      Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4      in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5      tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6      purpose-built trajectory model localises it far more precisely, giving cleaner
7      candidate rallies than person-velocity heuristics.
8
9      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11     TrackNet runner and identity.
12     """
13
14     from typing import Any, Dict, Tuple
15
16     from backend.config.models import IndexingConfig
17     from backend.pipeline.detectors.base import DetectorRunner
18     from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
19     from backend.pipeline.segmenters.base import segmenter_registry
20     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
21
22
23     class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "tracknet"
25         display_label = "TrackNet"
26
27         @property
28         def name(self) -> str:
29             return "tracknet_hybrid"
30
31         @property
32         def description(self) -> str:
33             return (
34                 "Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
35                 "candidate rallies + AI provider for high-precision semantic boundaries."
36             )
37
38         def _build_runner(
39             self, idx_cfg: IndexingConfig
40         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
41             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
42             return TrackNetRunner(cfg), {
43                 "weights_path": cfg.weights_path,
44                 "queue_length": cfg.queue_length,
45             }
46
47
48     segmenter_registry.register(TrackNetHybridSegmenter)
49
```

19. user (2026-07-07T15:20:31.220Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
```

```

6         1. Translate the Windows video path to its WSL-visible form (and optionally
7           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import csv
18    import logging
19    import os
20    import shlex
21    import subprocess
22    from dataclasses import dataclass
23    from typing import Any, Dict, List, Optional, Tuple
24
25    from backend.config.models import IndexingConfig
26    from backend.pipeline.detectors.base import (
27        DetectorRunner,
28        WslCommandMixin,
29        wsl_tilde_quote,
30    )
31
32    logger = logging.getLogger(__name__)
33
34
35    @dataclass
36    class TrackNetConfig:
37        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
38
39        distro: str = "Ubuntu"
40        repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
41        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
42        conda_env: str = "TrackNetV4"
43        weights_path: str = "" # WSL path to the .h5/.keras weights (REQUIRED)
44        queue_length: int = 5
45        stage_in_wsl: bool = True # copy video into WSL fs first (perf)
46        wsl_stage_dir: str = "~/clips" # where staged videos/outputs live in WSL
47        timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no timeout)
48        keep_frames: bool = False # retain staged video after success (debug only)
49        min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
50
51    @classmethod
52    def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "TrackNetConfig":
53        if isinstance(idx_cfg, dict):
54            idx_cfg = IndexingConfig(**idx_cfg)
55        tn = idx_cfg.tracknet
56        return cls(
57            distro=tn.wsl_distro,
58            repo_dir=tn.repo_dir,
59            conda_sh=tn.conda_sh,
60            conda_env=tn.conda_env,
61            weights_path=tn.weights_path,
62            queue_length=tn.queue_length,
63            stage_in_wsl=tn.stage_in_wsl,
64            wsl_stage_dir=tn.wsl_stage_dir,
65            timeout_sec=tn.timeout_sec,
66            keep_frames=tn.keep_frames,
67            min_free_gb=tn.min_free_gb,
68        )

```

```

69
70
71 def to_wsl_mnt_path(win_path: str) -> str:
72     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
73
74     Already-POSIX paths (starting with / or ~) are returned unchanged so the
75     same helper is safe to call on either kind of path.
76     """
77     p = str(win_path)
78     if p.startswith("/") or p.startswith("~"):
79         return p
80     p = p.replace("\\", "/")
81     if len(p) >= 2 and p[1] == ":":
82         drive = p[0].lower()
83         return f"/mnt/{drive}{p[2:]}"
84     return p
85
86
87 @dataclass
88 class TrajectoryPoint:
89     frame: int
90     visible: bool
91     x: float
92     y: float
93
94
95 class TrackNetRunner(WslCommandMixin, DetectorRunner):
96     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
97
98     Implements the common DetectorRunner contract so a future Linux-native or
99     alternative trajectory runner can be substituted without touching the hybrids.
100    """
101
102    def __init__(self, cfg: TrackNetConfig):
103        self.cfg = cfg
104
105    def healthcheck(self) -> Tuple[bool, str]:
106        """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108        Returns (ok, message). Cheap to call before a long run.
109        """
110        cmd = (
111            f"conda activate {self.cfg.conda_env} && "
112            f'python -c "import tensorflow as tf; '
113            f'print('TF', tf.__version__, 'GPUs',
114            len(tf.config.list_physical_devices('GPU')))"
115        )
116        try:
117            res = self._wsl_bash(cmd)
118        except FileNotFoundError:
119            return (
120                False,
121                "`wsl` not found ? is this running on Windows with WSL2 installed?",
122            )
123        except subprocess.TimeoutExpired:
124            return False, "WSL healthcheck timed out."
125        out = (res.stdout or "").strip()
126        if res.returncode != 0:
127            return False, f"WSL env check failed: {(res.stderr or out).strip()}"
128        return True, out
129
130    # ----- inference
131
132    def run_predict(
133        self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None

```

```

133 ) -> Optional[str]:
134     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
135
136     ``output_win_dir`` is a Windows directory (under the repo's output/) that
137     WSL writes into via its /mnt view, so the Windows process can read the CSV
138     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
139     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
140     a filename cannot collide on the output CSV.
141     """
142     if not self.cfg.weights_path:
143         logger.error(
144             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
145             "in config.json to the WSL path of your trained TrackNetV4 weights."
146         )
147         return None
148
149     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
150     # relative paths through unchanged, so WSL would resolve them against the
151     # predict.py cwd instead of the repo (same bug class as wasb_runner).
152     output_win_dir = os.path.abspath(output_win_dir)
153     os.makedirs(output_win_dir, exist_ok=True)
154     # Normalize separators so basename/splitext work when tests (or collector)
155     # pass Windows-style paths on a Linux host.
156     video_win_path = str(video_win_path).replace("\\", "/")
157     base = os.path.basename(video_win_path)
158     stem, ext = os.path.splitext(base)
159
160     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
161     if ext.lower() not in (".mp4", ".avi"):
162         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
163         return None
164
165     wsl_out = to_wsl_mnt_path(output_win_dir)
166     # Namespace by video_id so two same-filename videos don't collide on the CSV.
167     # predict.py derives its output name from the (staged) video stem, so staging
168     # under the namespaced name propagates into "<key>_predict.csv".
169     key = f"{stem}_{video_id[:12]}" if video_id else stem
170     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
171
172     # Resolve the video path WSL will read.
173     if self.cfg.stage_in_wsl:
174         staged = self._stage_video(video_win_path, f"{key}{ext}")
175         if staged is None:
176             return None
177         wsl_video = staged
178     else:
179         # No staging: predict.py reads /mnt directly and writes
180         # "<stem>_predict.csv".
181         # We cannot rename its output, so fall back to the un-namespaced name.
182         wsl_video = to_wsl_mnt_path(video_win_path)
183         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
184
185     # Disk guard (D11): BLOCK before extraction if the WSL stage fs can't hold ~15x
the source
186     # (frame PNGs balloon the footprint). Best-effort measuring ? an undeterminable
read never
187     # blocks; min_free_gb (config) still acts as an absolute floor. See
base._wsl_frame_stage_shortfall.
188     shortfall = self._wsl_frame_stage_shortfall(video_win_path)
189     if shortfall:
190         logger.error("TrackNet inference aborted ? %s", shortfall)
191         return None
192
193     cmd = (

```

```

193         f"conda activate {self.cfg.conda_env} && "
194         f"cd {self.cfg.repo_dir}/src && "
195         f"python predict.py "
196         f"--video_path {wsl_tilde_quote(wsl_video)} "
197         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
198         f"--output_dir {wsl_tilde_quote(wsl_out)} "
199         f"--queue_length {self.cfg.queue_length}"
200     )
201     logger.info("Running TrackNet inference on %s ...", base)
202     try:
203         res = self._wsl_bash(cmd)
204     except subprocess.TimeoutExpired:
205         logger.error(
206             "TrackNet inference timed out after %ss.", self.cfg.timeout_sec
207         )
208         return None
209
210     if res.returncode != 0:
211         logger.error(
212             "TrackNet predict.py failed:\n%s", (res.stderr or res.stdout)[-2000:]
213         )
214         return None
215
216     if not os.path.exists(expected_csv_win):
217         logger.error(
218             "TrackNet finished but expected CSV not found at %s", expected_csv_win
219         )
220         return None
221     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
222
223     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
224     # regenerable intermediate). On earlier failure we return above without
225     cleaning.
226     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
227         logger.info(
228             "Cache hygiene: removing staged video for %s "
229             "(set indexing.tracknet.keep_frames=true to retain).",
230             key,
231         )
232         self._wsl_rm_rf([wsl_video])
233     return expected_csv_win
234
235 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
236     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
237
238     Returns the WSL path to the staged copy, or None on failure.
239     """
240     src_mnt = to_wsl_mnt_path(video_win_path)
241     reason = self.wsl_source_unreadable_reason(src_mnt)
242     if reason:
243         logger.error("Cannot stage video into WSL: %s", reason)
244         return None
245     dest = f"{self.cfg.wsl_stage_dir}/{base}"
246     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
247     {wsl_tilde_quote(dest)} && echo OK"
248     try:
249         res = self._wsl_bash(cmd)
250     except subprocess.TimeoutExpired:
251         logger.error("Staging video into WSL timed out.")
252         return None
253     if res.returncode != 0 or "OK" not in (res.stdout or ""):
254         logger.error(
255             "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
256         )
257         return None

```

```

256         return dest
257
258     # ----- CSV -> windows
259
260     @staticmethod
261     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
262         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
263         points: List[TrajectoryPoint] = []
264         with open(csv_win_path, newline="") as f:
265             reader = csv.DictReader(f)
266             for row in reader:
267                 try:
268                     points.append(
269                         TrajectoryPoint(
270                             frame=int(row["Frame"]),
271                             visible=int(row["Visibility"]) == 1,
272                             x=float(row["X"]),
273                             y=float(row["Y"]),
274                         )
275                     )
276                 except (KeyError, ValueError):
277                     continue
278             points.sort(key=lambda p: p.frame)
279             return points
280
281     @staticmethod
282     def _active_frames(
283         points: List[TrajectoryPoint],
284         fps: float,
285         frame_width: float,
286         velocity_thresh: float,
287     ) -> List[Tuple[int, float]]:
288         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
289
290         Computes each visible point's normalised velocity from the last *visible* point;
291         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
292         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
293         the caller so identical floats feed the velocity math."""
294         active = [] # (frame_idx, velocity)
295         prev: Optional[TrajectoryPoint] = None
296         for p in points:
297             if not p.visible:
298                 prev = None # break the trajectory; absent shuttle = not in flight
299                 continue
300             if prev is not None:
301                 dframes = p.frame - prev.frame
302                 if dframes > 0:
303                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
304                     velocity = (dist / frame_width) * (fps / dframes)
305                     if velocity > velocity_thresh:
306                         active.append((p.frame, velocity))
307             prev = p
308         return active
309
310     @staticmethod
311     def trajectory_to_action_windows(
312         points: List[TrajectoryPoint],
313         fps: float,
314         frame_width: float,
315         chunk_start: float = 0.0,
316         velocity_thresh: float = 0.02,
317         merge_gap: float = 2.0,

```

```

318         min_window_duration: float = 2.0,
319         chunk_index: Optional[int] = None,
320         max_window_duration: float = 0.0,
321         min_split_gap: float = 0.5,
322         window_hard_cap: bool = False,
323         window_inactivity_end: float = 0.0,
324         inactivity_min_density: float = 0.34,
325         inactivity_rally_thresh: float = 0.08,
326         inactivity_temporal_density: bool = False,
327         window_blob_fallback: bool = False,
328         window_blob_factor: float = 4.0,
329     ) -> List[Dict[str, Any]]:
330         """Convert a shuttle trajectory into candidate rally windows.
331
332         A frame is "active" when the shuttle is visible AND its inter-frame
333         displacement (normalised by frame width, per second) exceeds
334         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
335         ``merge_gap`` seconds of each other are grouped into one window; short
336         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
337         shape feeding the AI-handoff phase is identical.
338
339         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
340         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
341         progress" and a single window swallows several rallies + the dead time between them. When
342         set, any window longer than this is recursively SPLIT at its largest internal inactivity
343         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
344         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
345         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
346         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
347
348         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
349         an over-long window has NO internal inactivity gap to split on (a continuously-
350         active trajectory ? the shuttle never stops moving, e.g. the real-footage
351         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
352         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
353         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
354
355         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
356         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
357         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4
358         inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
359         ``window_blob_factor`` x ``max_window_duration`` once the gap-split and the principled rally-end trim
360         have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
361         is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
362         logged, and each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
363         that needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts

```

```

(recall
364         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
365
366         """
367         if fps <= 0:
368             fps = 30.0
369         if frame_width <= 0:
370             frame_width = 1280.0
371
372         active = TrackNetRunner._active_frames(
373             points, fps, frame_width, velocity_thresh
374         )
375
376         if not active:
377             return []
378
379         max_gap_frames = int(merge_gap * fps)
380
381         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
382             vels = [v for _, v in group]
383             return {
384                 "start": group[0][0] / fps + chunk_start,
385                 "end": group[-1][0] / fps + chunk_start,
386                 "active_frame_count": len(group),
387                 "peak_velocity": max(vels),
388                 "mean_velocity": sum(vels) / len(vels),
389                 "track_id_count": 1, # single shuttle; kept for window-shape parity
390                 "chunk_index": chunk_index,
391             }
392
393         groups: List[List[Tuple[int, float]]] = []
394         group = [active[0]]
395         for af in active[1:]:
396             if af[0] - group[-1][0] <= max_gap_frames:
397                 group.append(af)
398             else:
399                 groups.append(group)
400                 group = [af]
401         groups.append(group)
402
403         if max_window_duration > 0:
404             groups = TrackNetRunner._split_overlong_groups(
405                 groups,
406                 int(max_window_duration * fps),
407                 int(min_split_gap * fps),
408                 hard_cap=window_hard_cap,
409             )
410
411         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
412         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
413         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
414         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
415         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
416
417         if window_inactivity_end > 0:
418             tw = int(window_inactivity_end * fps)
419             groups = [
420                 TrackNetRunner._trim_inactive_tail(
421                     g,
422                     tw,
423                     inactivity_min_density,

```

```

422             inactivity_rally_thresh,
423             temporal_density=inactivity_temporal_density,
424         )
425         for g in groups
426     ]
427
428     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
429     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
430     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
431     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
432     forced_flags = [False] * len(groups)
433     if window_blob_fallback and max_window_duration > 0:
434         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
435             groups, int(max_window_duration * fps), window_blob_factor
436         )
437
438     windows = []
439     for g, forced in zip(groups, forced_flags):
440         w = _finalize(g)
441         if forced:
442             w["forced_cut"] = True
443             windows.append(w)
444     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
445
446     @staticmethod
447     def _trim_inactive_tail(
448         group: List[Tuple[int, float]],
449         trail_frames: int,
450         min_density: float,
451         rally_thresh: float,
452         temporal_density: bool = False,
453     ) -> List[Tuple[int, float]]:
454         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
455         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
456         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
457         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
458         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
459         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
460         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
461         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
462         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).
463
464         ``temporal_density`` (default False = legacy/promoted behaviour) selects the
density
465         DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
the legacy
466         denominator ? the COUNT of active samples in the trailing window (``i - lo +
1``) ? reads a
467         lone fast sample after a long tracking gap as 100% dense and holds the tail
open. When True,
468         the denominator is the trailing window's TEMPORAL length instead, so sparse
tracking can't
469         inflate density. With DENSE tracking the two are identical, so this is a no-op

```

```

on
470         well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
golden set
471         at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
472         if trail_frames <= 0 or len(group) < 2:
473             return group
474         frames = [f for f, _ in group]
475         fast = [1 if v > rally_thresh else 0 for _, v in group]
476         lo = 0
477         fast_sum = 0
478         last_dense = -1
479         for i in range(len(frames)):
480             fast_sum += fast[i]
481             while frames[lo] <= frames[i] - trail_frames:
482                 fast_sum -= fast[lo]
483                 lo += 1
484             if temporal_density:
485                 # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
486                 # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
487                 denom = min(trail_frames, frames[i] - frames[0] + 1)
488             else:
489                 denom = (
490                     i - lo + 1
491                 ) # legacy: count of active samples in the trailing window
492             if denom > 0 and fast_sum / denom >= min_density:
493                 last_dense = i
494         return group[: last_dense + 1] if last_dense >= 0 else group
495
496     @staticmethod
497     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
498         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
499         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
500         mid = (group[0][0] + group[-1][0]) / 2.0
501         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
502         return split_i
503
504     @staticmethod
505     def _split_overlong_groups(
506         groups: List[List[Tuple[int, float]]],
507         max_win_frames: int,
508         min_split_frames: int,
509         hard_cap: bool = False,
510     ) -> List[List[Tuple[int, float]]]:
511         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
512         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
513         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
514         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
515         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
516         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
517         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
518         if max_win_frames <= 0:
519             return groups
520         out: List[List[Tuple[int, float]]] = []

```

```

521         work = list(groups)
522         while work:
523             g = work.pop()
524             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
525                 out.append(g)
526                 continue
527             best_i, best_gap = -1, -1
528             for i in range(1, len(g)):
529                 gap = g[i][0] - g[i - 1][0]
530                 if gap > best_gap:
531                     best_gap, best_i = gap, i
532             if best_i < 0 or best_gap < min_split_frames:
533                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
534                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
535                 if hard_cap:
536                     split_i = TrackNetRunner._midpoint_split_index(g)
537                     work.append(g[:split_i])
538                     work.append(g[split_i:])
539                 else:
540                     out.append(g)
541                     continue
542                 work.append(g[:best_i])
543                 work.append(g[best_i:])
544             out.sort(key=lambda grp: grp[0][0])
545             return out
546
547     @staticmethod
548     def _blob_fallback_chop(
549         groups: List[List[Tuple[int, float]]],
550         max_win_frames: int,
551         blob_factor: float = 4.0,
552     ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
553         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
554         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
555         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
556         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
557         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
558         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
559         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
560         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
561         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
562         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
563         that piece came from a forced chop. Iterative worklist to bound stack depth."""
564         if max_win_frames <= 0:
565             return groups, [False] * len(groups)
566         blob_frames = max_win_frames * max(1.0, blob_factor)
567         out: List[List[Tuple[int, float]]] = []
568         flags: List[bool] = []
569         forced = 0
570         for g in groups:
571             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
572                 out.append(g)
573                 flags.append(False)
574                 continue

```

```

575         pieces: List[List[Tuple[int, float]]] = []
576         work = [g]
577         while work:
578             h = work.pop()
579             if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
580                 pieces.append(h)
581                 continue
582             split_i = TrackNetRunner._midpoint_split_index(h)
583             work.append(h[:split_i])
584             work.append(h[split_i:])
585             pieces.sort(key=lambda grp: grp[0][0])
586             out.extend(pieces)
587             flags.extend([True] * len(pieces))
588             forced += len(pieces) - 1
589         if forced:
590             logger.warning(
591                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
592                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
593                 "motion), not a bigger cap",
594                 forced,
595             )
596         return out, flags
597

```

20. assistant (2026-07-07T15:21:25.193Z)

Now let me read the trajectory base (to see what a failed healthcheck actually does, and whether tracknet's `\_build\_runner` override bypasses `detector\_impl`), the config model, and the served config.

21. user (2026-07-07T15:21:26.020Z)

```

1         """Shared engine for trajectory-substrate hybrid segmenters.
2
3         tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4         standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5         now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10        A concrete subclass supplies only its identity and runner wiring:
11        - ``family`` ? config section under ``indexing.*``, output subdir, and
12          the ``<family>-hybrid-<model>`` detector tag in metadata.
13        - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14        - ``name`` / ``description`` properties (the registry contract).
15        - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16          detection_params extras such as weights_path).
17
18        This base is deliberately NOT registered; only concrete subclasses register.
19        """
20
21        import concurrent.futures
22        import logging
23        import os
24        import time
25        from typing import Any, Dict, List, Optional, Tuple, Union
26
27        import cv2
28
29        from backend.config.models import IndexingConfig
30        from backend.pipeline.detectors.base import DetectorRunner
31        from backend.pipeline.segmenters.base import BasePlaySegmenter

```

```

32     from backend.providers import provider_registry
33
34     # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35     from backend.results import Failure
36     from backend.sports import sport_registry
37     from backend.utils.paths import output_base
38     from backend.utils.video import extract_video_chunk, get_video_duration
39
40     logger = logging.getLogger(__name__)
41
42
43     def _fmt_ts(seconds: float) -> str:
44         seconds = max(0, int(seconds))
45         h, rem = divmod(seconds, 3600)
46         m, s = divmod(rem, 60)
47         return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50     class TrajectoryHybridSegmenter(BasePlaySegmenter):
51         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
52
53         #: config section under `indexing` (velocity_threshold lives there), the
54         #: output subdir for trajectories, and the metadata detector-tag prefix.
55         family: str = ""
56         #: human-readable label used in log lines.
57         display_label: str = ""
58
59         def _build_runner(
60             self, idx_cfg: IndexingConfig
61         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
62             """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
63             raise NotImplementedError
64
65         def _build_native_runner(
66             self, idx_cfg: IndexingConfig
67         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
68             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
69             native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
70             raise NotImplementedError(
71                 f"detector_impl='native' is not supported for the "
72                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
73                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
74             )
75
76         def _resolve_runner(
77             self, idx_cfg: IndexingConfig
78         ) -> Tuple[DetectorRunner, Dict[str, Any]]:
79             """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
80
81             ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
82             ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
83             is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
84             in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85             ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
86             (the WSL runner) so the default production path is unchanged

```

```

87         (`docs/DECOUPLED_COMPUTE/`).
88         """
89         # Safety net: accept raw dicts from legacy / test callers.
90         if isinstance(idx_cfg, dict):
91             idx_cfg = IndexingConfig(**idx_cfg)
92         impl = (
93             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94         ).lower()
95         if impl == "stub":
96             from backend.pipeline.detectors.stub_runner import (
97                 StubConfig,
98                 StubDetectorRunner,
99             )
100
101             logger.info(
102                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103                 self.display_label or "trajectory",
104             )
105             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
106                 "detector_impl": "stub"
107             }
108         if impl in ("native", "native_wasb", "wasb_native"):
109             logger.info(
110                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
111                 self.display_label or "trajectory",
112             )
113             return self._build_native_runner(idx_cfg)
114         return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125     @staticmethod
126     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127         """Provider-reported exchange count, else a duration-based estimate.
128
129         One canonical implementation ? the twins had drifted (wasb relied on
130         ``int(None)`` raising into the fallback; same result, by accident).
131         """
132         shot_count = segment.get("shuttle_exchange_count")
133         if shot_count is None:
134             return max(3, int(duration / 0.8))
135         try:
136             return int(shot_count)
137         except (ValueError, TypeError):
138             return max(3, int(duration / 0.8))
139
140     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
141     -----
142     def _enrich_candidate_stats(
143         self,
144         cw: Dict[str, Any],
145         points: List[Any],
146         fps: float,
147         frame_width: float,
148         video_path: str,
149         idx_cfg: "IndexingConfig",
150     ) -> Dict[str, Any]:
151         """Stats dict persisted into ``candidate_segments.stats`` for one window. The

```

```

BASE
151         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
152         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
153         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
154         return {
155             "peak_velocity": cw["peak_velocity"],
156             "mean_velocity": cw["mean_velocity"],
157             "active_frame_count": cw["active_frame_count"],
158             "track_id_count": cw["track_id_count"],
159         }
160
161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
170         Persisted candidates are NOT affected ? they remain the full superset for
offline
171         re-scoring."""
172         return list(range(len(windows)))
173
174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,
179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":
181         # Result contract (the ABC declares Success|Failure). Anything that means the
run could
182         # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
183         # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
184         # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
185         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
187         # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
188         label = self.display_label
189         # --- Sport profile + AI provider wiring (identical across segmenters) ---
190         sport_name = self.config.sport
191         try:
192             sport_profile = sport_registry.get(sport_name)
193         except KeyError as e:
194             logger.error(str(e))
195             # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
196             return Failure(str(e))
197
198         idx_cfg = self.config.indexing
199         skip_ai = bool(idx_cfg.skip_ai_handoff)
200         provider_name = self.config.ai_provider
201         if skip_ai:
202             model_name = "local-noai"
203             logger.info(
204                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "

```

```

205         "be emitted as rallies; no %s handoff, no API key needed.",
206         label,
207         provider_name,
208     )
209 else:
210     model_name = self.config.ai_model
211     api_key_env_var = f"{provider_name.upper()}_API_KEY"
212     api_key = os.environ.get(api_key_env_var)
213     if not api_key:
214         from backend.pipeline.segmenters._keyhint import api_key_hint
215
216         msg = (
217             f"{api_key_env_var} environment variable is not set! "
218             f"To run the {provider_name} handoff, set your API key. "
219             + api_key_hint(api_key_env_var)
220         )
221         logger.error(msg)
222         # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
223         # marked failed instead of a silent empty "success" (mirrors gemini; the
worker's
224         # own 0-segment diagnostic names this missing-key case as a top cause).
225         return Failure(msg, is_recoverable=True)
226     try:
227         provider = provider_registry.get(
228             provider_name, api_key=api_key, model_name=model_name
229         )
230     except KeyError as e:
231         logger.error(str(e))
232         return Failure(str(e))
233
234     video_duration = get_video_duration(video_path)
235     if video_duration <= 0:
236         logger.error("Invalid video duration.")
237         # An unreadable/broken video, not a 0-rally match.
238         return Failure(
239             message=f"{label}: invalid/unreadable video duration
({video_duration}).",
240         )
241     fps, frame_width = self._probe_fps_width(video_path)
242
243     # --- Runner config + windowing thresholds ---
244     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
245     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246     # it delegates to the subclass _build_runner (the real WSL/native runner).
247     try:
248         runner, runner_params = self._resolve_runner(idx_cfg)
249     except NotImplementedError as e:
250         # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
251         # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
252         # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
success.
253         logger.error("%s: %s", label, e)
254         return Failure(message=f"{label}: {e}")
255
256     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
257     merge_gap = idx_cfg.dead_time_merge_gap
258     min_window_duration = idx_cfg.min_action_window_duration
259     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
260     max_window_duration = idx_cfg.max_window_duration
261     window_min_split_gap = idx_cfg.window_min_split_gap
262     window_hard_cap = idx_cfg.window_hard_cap

```

```

263     window_inactivity_end = idx_cfg.window_inactivity_end
264     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
265     inactivity_min_density = idx_cfg.inactivity_min_density
266     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
267     window_blob_fallback = idx_cfg.window_blob_fallback
268     window_blob_factor = idx_cfg.window_blob_factor
269     handoff_padding = idx_cfg.ai_handoff_padding
270     ai_max_workers = idx_cfg.ai_max_workers
271     log_candidates = idx_cfg.log_yolo_candidates
272     store_candidates = idx_cfg.store_yolo_candidates
273
274     detection_params = {
275         "detector": self.name,
276         "velocity_thresh": velocity_thresh,
277         "merge_gap": merge_gap,
278         "min_action_window_duration": min_window_duration,
279         **runner_params,
280     }
281
282     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
283     ok, msg = runner.healthcheck()
284     if not ok:
285         logger.error("%s detector environment not ready: %s", label, msg)
286         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
287         # failed/retryable instead of writing an empty-but-"successful" result.
288         return Failure(
289             message=f"{label} detector environment not ready: {msg}",
290             is_recoverable=True,
291         )
292     logger.info("%s detector env OK: %s", label, msg)
293
294     # --- Phase 2: trajectory -> candidate rally windows ---
295     # Trajectory detectors process the whole video in one pass (they carry
296     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
297     t_start = time.time()
298     out_dir = os.path.join(output_base(self.config), self.family)
299     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300     if not csv_path:
301         logger.error("%s produced no trajectory; aborting.", label)
302         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
304         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305         # rather than a silent 0-segment "success" (the wf294 incident).
306         return Failure(
307             message=(
308                 f"{label} detector produced no trajectory (timed out or aborted) ? "
309                 "detector failure, not a 0-rally video."
310             ),
311             is_recoverable=True,
312         )
313
314     points = runner.parse_trajectory_csv(csv_path)
315     logger.info(
316         "Parsed %d trajectory points (%d visible).",
317         len(points),
318         sum(1 for p in points if p.visible),
319     )
320
321     all_candidate_windows = runner.trajectory_to_action_windows(
322         points,
323         fps=fps,

```

```

324         frame_width=frame_width,
325         chunk_start=0.0,
326         velocity_thresh=velocity_thresh,
327         merge_gap=merge_gap,
328         min_window_duration=min_window_duration,
329         chunk_index=0,
330         max_window_duration=max_window_duration,
331         min_split_gap=window_min_split_gap,
332         window_hard_cap=window_hard_cap,
333         window_inactivity_end=window_inactivity_end,
334         inactivity_rally_thresh=inactivity_rally_thresh,
335         inactivity_min_density=inactivity_min_density,
336         inactivity_temporal_density=inactivity_temporal_density,
337         window_blob_fallback=window_blob_fallback,
338         window_blob_factor=window_blob_factor,
339     )
340     logger.info(
341         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
342         label,
343         time.time() - t_start,
344         len(all_candidate_windows),
345     )
346
347     if log_candidates:
348         for cw in all_candidate_windows:
349             logger.info(
350                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
351                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
352                 f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
353             )
354
355     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
356     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
357     # persistence and the handoff gate below.
358     window_stats = [
359         self._enrich_candidate_stats(
360             cw, points, fps, frame_width, video_path, idx_cfg
361         )
362         for cw in all_candidate_windows
363     ]
364
365     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
366     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
367     if store_candidates:
368         for i, cw in enumerate(all_candidate_windows):
369             candidate_ids[i] = self.db.add_candidate_segment(
370                 video_id,
371                 detector=self.name,
372                 start_time=cw["start"],
373                 end_time=cw["end"],
374                 chunk_index=cw["chunk_index"],
375                 confidence=min(1.0, cw["peak_velocity"]),
376                 stats=window_stats[i],
377                 detection_params=detection_params,
378             )
379
380     if not all_candidate_windows:
381         return []
382
383     # --- Phase 3: AI provider handoff over padded candidate clips ---
384     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted

```

```

385 # candidates above stay the fullsuperset ? only the served play_segments are
gated.
386 handoff_indices = self._select_for_handoff(
387     all_candidate_windows, window_stats, idx_cfg
388 )
389
390 ai_segments: List[dict] = []
391 if skip_ai:
392     # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
393     # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
394     # falls back to the duration-based estimate (no AI exchange count).
395     ai_segments = [
396         {
397             "start_time": all_candidate_windows[wi]["start"],
398             "end_time": all_candidate_windows[wi]["end"],
399             "shuttle_exchange_count": None,
400             "shot_count_confidence": "LocalNoAI",
401             "_candidate_id": candidate_ids[wi],
402         }
403         for wi in handoff_indices
404     ]
405     logger.info(
406         "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
407         "(no AI handoff).",
408         len(ai_segments),
409     )
410 else:
411     handoff_dir = os.path.join(
412         output_base(self.config), f"chunks_{provider_name}_handoff"
413     )
414     os.makedirs(handoff_dir, exist_ok=True)
415     logger.info(
416         "Phase 3: %s validation on %d candidate windows...",
417         provider_name,
418         len(handoff_indices),
419     )
420
421     def process_candidate_window(
422         wi: int, window: Dict[str, Any]
423     ) -> Tuple[List[dict], bool]:
424         """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
425         verdict (clip extraction failed, or the provider returned
metrics['error'])."""
426         w_start = max(0, window["start"] - handoff_padding)
427         w_end = min(video_duration, window["end"] + handoff_padding)
428         w_dur = w_end - w_start
429         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
430         if not extract_video_chunk(
431             video_path, w_start, w_dur, w_path, reencode=True
432         ):
433             return [], True # extraction failed ? no verdict for this window
434         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
435         segments, metrics = provider.analyze_video(
436             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
437         )
438         # B2: a provider failure returns [] with metrics["error"] set ? that is
439         # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
440         # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
441         errored = bool(metrics.get("error"))

```

```

442         if errored:
443             logger.warning(
444                 "[Handoff %d] provider error ? window %d skipped: %s",
445                 wi + 1,
446                 wi,
447                 metrics["error"],
448             )
449         valid = []
450         for seg in segments:
451             try:
452                 seg["start_time"] = float(seg["start_time"]) + w_start
453                 seg["end_time"] = float(seg["end_time"]) + w_start
454                 seg["_candidate_id"] = candidate_ids[wi]
455                 valid.append(seg)
456             except (ValueError, TypeError, KeyError) as e:
457                 logger.warning(
458                     "Dropping segment with unparseable timestamps: %s ? %s",
459                     seg,
460                     e,
461                 )
462         try:
463             os.remove(w_path)
464         except OSError:
465             pass
466         return valid, errored
467
468     with concurrent.futures.ThreadPoolExecutor(
469         max_workers=ai_max_workers
470     ) as executor:
471         futures = [
472             executor.submit(
473                 process_candidate_window, wi, all_candidate_windows[wi]
474             )
475             for wi in handoff_indices
476         ]
477         window_results = [future.result() for future in futures]
478     for valid, _errored in window_results:
479         ai_segments.extend(valid)
480     handoff_failed = sum(1 for _valid, errored in window_results if errored)
481     try:
482         os.rmdir(handoff_dir)
483     except OSError:
484         pass
485     logger.info(
486         "Phase 3 completed. AI confirmed %d semantic play segments.",
487         len(ai_segments),
488     )
489     # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
490     # (provider error / clip extraction failed), the run has zero information ?
491     # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
492     # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
493     if handoff_indices and handoff_failed == len(handoff_indices):
494         msg = (
495             f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
496             "window(s) errored (provider error / clip extraction failed) ?
incomplete "
497             "run, not a 0-rally match."
498         )
499         logger.error(msg)
500         return Failure(message=msg, is_recoverable=True)

```

```

501
502     # --- Phase 4: DB population + candidate linking ---
503     segments_data = []
504     for segment in ai_segments:
505         try:
506             start = float(segment["start_time"])
507             end = float(segment["end_time"])
508             dur = end - start
509             metadata = {
510                 "shot_count": self._shot_count_from(segment, dur),
511                 "shot_count_confidence": segment.get(
512                     "shot_count_confidence", "Unknown"
513                 ),
514                 "detector": f"{self.family}-hybrid-{model_name}",
515             }
516             segment_id = self.db.add_play_segment(
517                 video_id, start, end, None, None, metadata
518             )
519             if segment_id:
520                 candidate_id = segment.get("_candidate_id")
521                 if candidate_id is not None:
522                     self.db.link_candidate_to_segment(candidate_id, segment_id)
523                 segments_data.append(
524                     {
525                         "id": segment_id,
526                         "start_time": start,
527                         "end_time": end,
528                         "duration": dur,
529                         "metadata": metadata,
530                     }
531                 )
532             except Exception as e:
533                 logger.warning("Skipping invalid segment struct: %s", e)
534
535     return segments_data
536

```

22. assistant (2026-07-07T15:22:47.116Z)

Now I have the critical behavioral facts. Let me verify the config defaults (does `indexing.tracknet.weights\_path` default to empty?) and the actual served config on the hosted control-plane, to assess whether the scenario is reachable.

23. user (2026-07-07T15:22:50.372Z)

```

46:class TrackNetConfig(BaseModel):
47:    weights_path: str = ""
48:
49:class WasbIndexConfig(BaseModel):
50:    """Typed config for the live WASB shuttle substrate (indexing.wasb).
51:    weights_path: str = (
52:        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
53:
54:168:class FusionConfig(BaseModel):
55:    # Which trajectory substrate seeds the windows (shuttle stays primary).
56:    substrate: Literal["wasb", "tracknet"] = "wasb"
57:    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
58:202:class IndexingConfig(BaseModel):
59:    detector_impl: str = "auto"
60:
61:463:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.

```

24. user (2026-07-07T15:22:52.662Z)

```

=== config.json tracknet block ===
44:     "tracknet": {
49:         "weights_path": "",

```

```

56:         "wasb": {
60:             "conda_env": "wasb",
61:             "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",

```

```

=== deploy configs referencing tracknet weights ===
=== any served/deploy config.json ===
cloudrun
digitalocean
gcp

```

25. assistant (2026-07-07T15:23:38.316Z)

Confirmed the key facts: `TrackNetConfig.weights\_path` defaults to `""` (models.py:51), the repo `config.json` sets `indexing.tracknet.weights\_path = ""` (line 49), and `FusionConfig.substrate` defaults to `"wasb"` (line 180). Let me verify the hosted deploy config doesn't set a tracknet weights path or override detector_impl, to close the reachability question.

26. user (2026-07-07T15:23:42.277Z)

```

=== deploy/cloudrun listing ===
deploy/cloudrun:
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
post_deploy_smoke.ps1
README.md

```

```

=== any tracknet/weights/detector_impl/RALLY_DETECTOR_IMPL across deploy ===
deploy/cloudrun/Dockerfile:18:# mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before
any public, non-owner serving.
deploy/cloudrun/Dockerfile:111:# runtime (WASB-C3). detector_impl=native = the Linux-native WASB
runner; cuda stays the default.
deploy/cloudrun/Dockerfile:118:     WASB_WEIGHTS=/opt/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
deploy/cloudrun/Dockerfile:119:     RALLY_DETECTOR_IMPL=native
deploy/cloudrun/post_deploy_smoke.ps1:37:  "--set=indexing.detector_impl=native",
deploy/cloudrun/README.md:37:  --set-env-vars
WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
deploy/cloudrun/README.md:146:  + `indexing.detector_impl=native` in the per-execution args so
the Job never re-dispatches itself.
deploy/cloudrun/README.md:166:  `indexing.detector_impl=native`.
deploy/digitalocean/gpu_setup.sh:4:# real trajectory extraction. Replicates
docs/TRACKNET_WSL_SETUP.md's WSL conda env
deploy/digitalocean/README.md:69:  RALLY_DETECTOR_IMPL=stub python -m pytest
tests/test_stub_runner.py -q`.
deploy/digitalocean/README.md:73:- **M3.5 WSL?native detector parity** ? replicate the
`TRACKNET_WSL_SETUP.md` WASB conda env *natively*
deploy/gcp/nightly_regression/run_on_vm.sh:94:# runner env. WASB_WEIGHTS alone is NOT enough ?
the native runner also needs WASB_REPO_DIR and the
deploy/gcp/nightly_regression/run_on_vm.sh:100:  export WASB_WEIGHTS=$HOME/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
deploy/gcp/README.md:192:Point the native runner at them: `export WASB_WEIGHTS=~/.models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
deploy/gcp/README.md:194:`docs/TRACKNET_WSL_SETUP.md` for the WASB env. **WASB-C3:** weight
provenance is an open legal item
deploy/gcp/README.md:205:  "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl":
"native" },
deploy/gcp/README.md:215:RALLY_DETECTOR_IMPL=native python -m backend.worker train \

```

27. user (2026-07-07T15:23:43.087Z)

```

=== main.py capabilities weights probe (approx 360-420) ===
def api_version():

```

```

    """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md P1)."""
    return {
        "engine_version": _engine_version(),
        "api_version": API_VERSION,
        "ui": _bundled_ui_version(),
    }

@app.get("/api/capabilities")
def capabilities(request: Request):
    """Honest probe of what THIS box can do ? the first-run wizard renders the compute step from
    it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT, never
    its
    value. Best-effort + never raises (the wizard degrades gracefully)."""
    cfg = request.app.state.config
    indexing = getattr(cfg, "indexing", None)
    deployment = getattr(cfg, "deployment", None)
    ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
    ffprobe = shutil.which(ffprobe_bin())
    # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
    wasb = getattr(indexing, "wasb", None)
    weights_path = (
        os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
    )
    # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately NOT
    # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-negative
    on a
    # real GPU box) and importing it would add seconds to this probe. The detector healthcheck
    is the
    # authority on whether GPU inference actually works; this is just the wizard's first-order
    signal.
    nvidia_smi = shutil.which("nvidia-smi")
    return {
        "api_version": API_VERSION,
        "engine_version": _engine_version(),
        "segmenters": sorted(segmenter_registry.list_available().keys()),
        "default_segmenter": getattr(indexing, "default_segmenter", None),
        "ffmpeg": {
            "ffmpeg": bool(ffmpeg),
            "ffprobe": bool(ffprobe),
            "ffmpeg_path": ffmpeg,
            "ffprobe_path": ffprobe,
        },
        "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
        "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
        "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
        **uploads_api.upload_caps(cfg), # #480 item 5: the SPA pre-flight reads max_upload_mb
        "deployment": {
            "profile": getattr(deployment, "profile", "") or "",
            "compute_target": getattr(deployment, "compute_target", "") or "",
            # item 3: can this server satisfy a per-request compute="cloud" dispatch? The SPA
            shows
            # the "run in the cloud" toggle only when true (else it's local-only).
            "cloud_available": _cloud_available(cfg),
        },
        # P3: which storage media THIS box can store INTO (local always; s3/gcs if SDK+config;
        # gdrive if a mount resolves). The P5 medium picker renders only usable backends. No
        secrets.
        "storage": storage_capabilities(
            getattr(cfg, "storage", None), local_free_path=output_base(cfg)
        ),
    }

def _friendly_video_name(filepath: str) -> str:

```

```

    """A human name for a video from its stored path/URI ? the filename stem
    (``~/match_2026.mp4`` ?
    ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI (rsplit on
    ``/``).
    Falls back to the whole ``filepath`` if there's no basename."""
    base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
    stem = os.path.splitext(base)[0]
    return stem or base or str(filepath or "")

```

28. assistant (2026-07-07T15:24:40.346Z)

I now have everything needed to adjudicate. Let me consolidate the ground truth.

The deploy findings are decisive. The hosted Cloud Run control-plane image (`deploy/cloudrun/Dockerfile:118-119`) sets `WASB_WEIGHTS=/opt/models/...` and `RALLY_DETECTOR_IMPL=native`, and `**no deploy config anywhere` sets `indexing.tracknet.weights_path`. Combined with the default (`models.py:51` ? ``"`) and the served `config.json:49` ("tracknet": {"weights_path": ""})`, on the real control-plane `_local_detector_weights_present(cfg, "tracknet_hybrid")` evaluates path=""` ? returns False` ? the guard **fires** and returns the actionable message.`

Let me trace the mechanism once more against the exact code to be airtight, then decide.

Key verified facts:

- `_local_detector_weights_present` tracknet branch (orchestration_new.py:179-187):` path = idx.tracknet.weights_path or ""` ; returns bool(path) and os.path.exists(...)` . With path=""` ?` False` .`
- No `native_tracknet_runner.py` exists;` TrackNetRunner` is WSL-based (tracknet_runner.py:95` ,` healthcheck` catches` FileNotFoundError` ?` (False, "wsl` not found ?")` at lines 117-121).`
- `tracknet_hybrid._build_runner` always builds the WSL TrackNetRunner` (tracknet_hybrid.py:38-45);` detector_impl='native'` routes to base _build_native_runner` ?` NotImplementedError` (trajectory_hybrid.py:65-74).`
- Both failure modes are `**caught**` and returned as a clean `Failure` (trajectory_hybrid.py:247-254 for native NotImplementedError` ; 283-291 for failed healthcheck) ? surfaced as status=failed` with a message, not an uncaught deep crash.`
- `/api/capabilities` (main.py) probes only wasb_weights_present` , never tracknet.`

29. user (2026-07-07T15:24:56.347Z)

Structured output provided successfully